

# 課題 2

祝心磊

10/16 2025

## 1 HDL

### 1.1 HalfAdder

```
1 CHIP HalfAdder {  
2     IN a, b;  
3     OUT sum,  
4         carry;  
5  
6     PARTS:  
7     And(a=a,b=b,out=carry);  
8     Xor(a=a,b=b,out=sum);  
9 }
```

### 1.2 FullAdder(HalfAdder\*2)

```
1 CHIP FullAdder {  
2     IN a, b, c;  
3     OUT sum,  
4         carry;  
5  
6     PARTS:  
7     HalfAdder(a=a,b=b,sum=sum1,carry=carry1);  
8     HalfAdder(a=sum1,b=c,sum=sum,carry=carry2);  
9     Or(a=carry1,b=carry2,out=carry);  
10  
11 }
```

### 1.3 FullAdder(without HalfAdder)

```
1 CHIP FullAdder {  
2     IN a, b, c;  
3     OUT sum,  
4         carry;  
5  
6     PARTS:
```

```

7   And(a=a,b=b,out=carry1);
8   Xor(a=a,b=b,out=sum1);
9   And(a=sum1,b=c,out=carry2);
10  Xor(a=sum1,b=c,out=sum);
11  Or(a=carry1,b=carry2,out=carry);
12 }
13

```

## 1.4 Add16

```

1  CHIP Add16 {
2      IN a[16], b[16];
3      OUT out[16];
4
5      PARTS:
6      HalfAdder(a=a[0],b=b[0],sum=out[0],carry=temp0);
7      FullAdder(a=a[1],b=b[1],c=temp0,sum=out[1],carry=temp1);
8      FullAdder(a=a[2],b=b[2],c=temp1,sum=out[2],carry=temp2);
9      FullAdder(a=a[3],b=b[3],c=temp2,sum=out[3],carry=temp3);
10     FullAdder(a=a[4],b=b[4],c=temp3,sum=out[4],carry=temp4);
11     FullAdder(a=a[5],b=b[5],c=temp4,sum=out[5],carry=temp5);
12     FullAdder(a=a[6],b=b[6],c=temp5,sum=out[6],carry=temp6);
13     FullAdder(a=a[7],b=b[7],c=temp6,sum=out[7],carry=temp7);
14     FullAdder(a=a[8],b=b[8],c=temp7,sum=out[8],carry=temp8);
15     FullAdder(a=a[9],b=b[9],c=temp8,sum=out[9],carry=temp9);
16     FullAdder(a=a[10],b=b[10],c=temp9,sum=out[10],carry=temp10);
17     FullAdder(a=a[11],b=b[11],c=temp10,sum=out[11],carry=temp11);
18     FullAdder(a=a[12],b=b[12],c=temp11,sum=out[12],carry=temp12);
19     FullAdder(a=a[13],b=b[13],c=temp12,sum=out[13],carry=temp13);
20     FullAdder(a=a[14],b=b[14],c=temp13,sum=out[14],carry=temp14);
21     FullAdder(a=a[15],b=b[15],c=temp14,sum=out[15],carry=temp15);
22 }

```

## 1.5 Incrementer

```

1  CHIP Inc16 {
2      IN in[16];
3      OUT out[16];
4
5      PARTS:
6      Add16(a=in,b[0]=true,out=out);
7  }

```

## 1.6 Alu

```

1  CHIP ALU {
2
3

```

```

4
5     IN
6         x[16], y[16],
7         zx, nx, zy, ny, f, no;
8     OUT
9         out[16], zr, ng;
10
11     PARTS:
12
13     Mux16(a=x, b=false, sel=zx, out=x1);
14     Not16(in=x1, out=notx);
15     Mux16(a=x1, b=notx, sel=nx, out=x2);
16
17     Mux16(a=y, b=false, sel=zy, out=y1);
18     Not16(in=y1, out=noty);
19     Mux16(a=y1, b=noty, sel=ny, out=y2);
20
21     Add16(a=x2, b=y2, out=addOut);
22     And16(a=x2, b=y2, out=andOut);
23     Mux16(a=andOut, b=addOut, sel=f, out=resultOut);
24
25     Not16(in=resultOut, out=notresult);
26     Mux16(a=resultOut, b=notresult, sel=no, out=out1);
27
28     And16(a=out1, b=true, out=out);
29
30     Or16Way(in=out1, out=zr1);
31     Not(in=zr1, out=zr);
32
33     IsNeg(in=out1, out=ng);
34
35 }

```

## 1.7 Alu without help chip

```

1 CHIP ALU {
2
3
4     IN
5         x[16], y[16],
6         zx, nx, zy, ny, f, no;
7     OUT
8         out[16], zr, ng;
9
10    PARTS:
11        // x の前処理
12        Mux16(a=x, b=false, sel=zx, out=x1);
13        Not16(in=x1, out=notx);
14        Mux16(a=x1, b=notx, sel=nx, out=x2);
15

```

```

16 // y の前処理
17 Mux16(a=y, b=false, sel=zy, out=y1);
18 Not16(in=y1, out=noty);
19 Mux16(a=y1, b=noty, sel=ny, out=y2);
20
21 // 演算実行
22 Add16(a=x2, b=y2, out=addOut);
23 And16(a=x2, b=y2, out=andOut);
24 Mux16(a=andOut, b=addOut, sel=f, out=resultOut);
25
26 // 出力後処理
27 Not16(in=resultOut, out=notresult);
28 Mux16(a=resultOut, b=notresult, sel=no, out=finalout);
29
30 // ゼロ判定のための複製出力
31 And16(a=finalout, b=true, out=out);
32
33 // zr
34 Mux16(a= resultOut, b= notresult, sel=no, out [15]=Zero, out [0..7]= 0Z1, out [8..15]=0Z2);
35
36 Or8Way(in=0Z1, out=Z1);
37 Or8Way(in=0Z2, out=Z2);
38 And(a=Z1, b=Z2, out=zr);
39
40
41
42 Or(a=Zero, b=false, out=zg);
43
44 }

```

## 1.8 helpchip

### 1.8.1 Or16Way

```

1 CHIP Or16Way {
2   IN in[16];
3   OUT out;
4
5   PARTS:
6   Or(a = in[0], b = in[1], out = o1);
7   Or(a = o1, b = in[2], out = o2);
8   Or(a = o2, b = in[3], out = o3);
9   Or(a = o3, b = in[4], out = o4);
10  Or(a = o4, b = in[5], out = o5);
11  Or(a = o5, b = in[6], out = o6);
12  Or(a = o6, b = in[7], out = o7);
13  Or(a = o7, b = in[8], out = o8);
14  Or(a = o8, b = in[9], out = o9);
15  Or(a = o9, b = in[10], out = o10);
16  Or(a = o10, b = in[11], out = o11);

```

```

17   Or(a = o11, b = in[12], out = o12);
18   Or(a = o12, b = in[13], out = o13);
19   Or(a = o13, b = in[14], out = o14);
20   Or(a = o14, b = in[15], out = out);
21 }

```

### 1.8.2 IsNeg

```

1  CHIP IsNeg {
2
3      IN in[16];
4      OUT out;
5
6      PARTS:
7      Or(a = in[15], b = false, out = out);
8  }

```

## 2 加算器および Alu の実装についての説明

### 2.1 add16

まず、HalfAdder に入力をし、FullAdder を繰り返し呼び出すことで、add16 を実装できた。ただし、オーバーフローを無視する。

### 2.2 Alu

x,y の演算をそれぞれ実装して、制御ビットを実装する。また、bliut chip がない chip を定義し直す。